

Performance Testing

Date	15 July 2026
Team ID	SWTID-2026-2749
Project Name	OptiCrop: Smart Agricultural Production Optimization engine
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Web Browser (Google Chrome) + Flask Development Server + Manual Testing
Type of Testing	Functional Testing & Performance Testing
Target Module / API	Crop Prediction Module (/predict)
Test Environment	Local System (Windows 10, Python 3.10, Flask)
Test Date	15 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Open Home Page	1	5	Homepage loads successfully
2	Predict Crop using valid inputs	1	10	Correct crop recommendation displayed
3	Multiple Predictions	1	30	Application responds consistently
4	Navigation between pages	1	15	All pages load without errors

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.8 sec	Pass	Prediction generated quickly
2	Response Time (Max)	< 5 seconds	1.5 sec	Pass	No noticeable delay
3	Throughput (Req/sec)	5 Req/sec	7 Req/sec	Pass	Handled repeated requests
4	Error Rate	< 1%	0%	Pass	No errors during testing
5	CPU Utilization	< 80%	35%	Pass	Stable CPU usage
6	Memory Utilization	< 80%	28%	Pass	Efficient memory usage

Step 4: Observations & Analysis

Key Findings

The OptiCrop application responded quickly to prediction requests and consistently generated accurate crop recommendations. The Random Forest model achieved high prediction accuracy while maintaining low response time. The user interface remained responsive throughout testing.

Bottlenecks Identified

No major bottlenecks were observed during testing. Minor delays occurred only during the initial loading of the Flask application.

Optimization Steps Taken

The Random Forest model was selected after comparing multiple Machine Learning algorithms. The trained model was stored using Pickle to reduce loading time, and the Flask application was organized into modular components for improved maintainability and performance.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

